

GitClear — User Guide

GitClear reclaims disk space by finding the files Git ignores in your repositories — build output, `node_modules`, `bin`, `obj`, logs, caches — and moving them to the Recycle Bin.

It asks **Git itself** which files are ignored, so what GitClear offers to delete is exactly what Git already considers disposable. Your tracked source code is never a candidate for deletion.

Before you start

Requirement	Why
Windows 10 or 11	GitClear is a Windows desktop app and uses the Windows Recycle Bin.
.NET 10 Desktop Runtime	The app is built on .NET 10 / WPF.
Git installed and on your <code>PATH</code>	GitClear runs <code>git</code> to determine what is ignored. If Git is missing you will see "Git was not found on your PATH. Install Git to scan repositories."

You do **not** need to run GitClear from inside a repository, and it never modifies your Git history, index, or `.git` folder.

Running GitClear

Launch `GitClear.App.exe`, or from the source tree:

```
dotnet run --project src/GitClear.App/GitClear.App.csproj
```

The window at a glance

-----+
| Folder: [C:\Projects] [Browse...] [Find repositories] [Stop] |
+-----+
| (busy bar - animates while working) |
+-----+
Repositories	Folders	Files in selected folder
MyApp	[] MyApp 1.2 GB (8,431)	[] Name Size
C:\Projects\..	[x] node_modules 980 MB	[] debug.log 4.1 MB
	[] src	[] output.tmp 118 KB
Tools	[] bin 44 MB (210)	
C:\Projects\..		
+-----+		
Selected for deletion: 8,641 files - 1.02 GB [Undo last delete] [Delete ->]		
+-----+		
8,431 ignored files - 1.2 GB		
+-----+

- **Folder** — the root folder to search for repositories. Type a path, or use **Browse...**
- **Repositories** (left) — every Git repository found under that folder.
- **Folders** (middle) — the folders containing ignored files, each with its total ignored size and file count.
- **Files in selected folder** (right) — the individual ignored files in whichever folder you have highlighted.
- **Selection summary** — a running total of what you have ticked.
- **Status bar** (bottom) — what just happened, or what is happening now.

The three panes are resizable: drag the dividers between them.

Walkthrough: reclaiming space

1. **Choose a folder to search.** Click **Browse...** and pick a folder that contains your repositories (for example `C:\Projects`). GitClear starts searching as soon as you pick — you do not need to press **Find repositories** as well.

You can also type or paste a path into the **Folder** box and then click **Find repositories**.

2. **Pick a repository.** Repositories appear in the left pane as they are found. Click one and GitClear immediately scans it for ignored files.

A repository marked (**worktree**) is a Git worktree or submodule — its `.git` is a pointer file rather than a folder. It behaves the same way.

3. **Review what was found.** The status bar reports the total, for example *"8,431 ignored files · 1.2 GB"*. Browse the **Folders** tree to see where the space is going, and click any folder to list its files on the right.
4. **Tick what you want cleared.** See *Selecting what to clear* below.
5. **Delete.** Click **Delete selected** → **Recycle Bin**, confirm the prompt, and GitClear moves everything to the Recycle Bin, then re-scans so the view reflects reality.
6. **Changed your mind?** Click **Undo last delete** straight away.

Reading the folder tree

This is the one concept worth understanding, because it changes how deletion behaves.

Each tree row shows the folder name, its total ignored size, and its ignored file count. Some folders also carry a **folder marker** (a small folder glyph) with the tooltip *"Entire folder is ignored — deleted as a unit"*:

Row	Meaning
node_modules (marked) 980 MB (12,004 files)	The entire folder is ignored. Git confirms nothing inside it is tracked, so GitClear deletes the folder as a single unit — fast, even with thousands of files.
src 4.2 MB (3 files)	A mixed folder. It holds tracked files <i>as well as</i> ignored ones. Only the individual ignored files are ever touched; the folder itself and your tracked code stay put.

A marked folder is all-or-nothing: it has no expandable contents, and selecting it shows no file list. You either delete the whole folder or none of it. This is deliberate — it is what makes clearing a large `node_modules` near-instant instead of grinding through tens of thousands of individual files.

Sizes use 1024-based units with Windows-style labels (`bytes` , `KB` , `MB` , `GB`).

If a repository has no ignored files at all, the tree stays empty and the status bar says *"No ignored files found in this repository."*

Selecting what to clear

Checkboxes work like Windows File Explorer's tri-state selection:

- **Tick a folder** to select everything ignored beneath it; its subfolders and files are ticked too.
- **Untick a folder** to clear the whole subtree.
- A folder shows an **indeterminate** box when only *some* of its contents are selected.
- **Tick individual files** in the right-hand pane for fine-grained control.
- Ticking the **top row** (the repository itself) selects every ignored file in the repository.

The summary line keeps a running total — *"Selected for deletion: 8,641 files · 1.02 GB"* — so you always know how much you are about to reclaim before committing. With nothing ticked it reads *"Nothing selected."* and the Delete button stays disabled.

Deleting

Delete selected → Recycle Bin asks for confirmation first:

Move to Recycle Bin

Move 8,641 ignored files (1.02 GB) to the Recycle Bin?

You can restore them with Undo, or from the Recycle Bin.

On confirming, GitClear moves everything to the Recycle Bin, then re-scans the repository so the tree and totals reflect what is actually left. Your selection resets. The status bar confirms: *"Moved 8,641 files to the Recycle Bin. Use Undo to restore."*

Anything already gone (deleted by a build in the meantime, say) is skipped rather than treated as an error.

Undo

Undo last delete restores the most recent deletion from the Recycle Bin to its original location, then re-scans.

Undo is deliberately limited, and the boundaries are worth knowing:

- **One level only.** It undoes the last deletion, not a history of them.
- **Forgotten when you switch repositories** or deselect the current one.

- **Forgotten when you close GitClear.** It is not saved between sessions.
- **Best-effort.** If Windows cannot restore an item automatically you will see "*Nothing could be restored automatically — check the Recycle Bin.*" The items are still in the Recycle Bin; restore them yourself from there.

Because every deletion goes to the Recycle Bin, you can always fall back to restoring from the Recycle Bin manually, even long after Undo has been forgotten.

Stopping a long operation

Stop cancels whatever is currently running — searching for repositories, scanning a repository, or deleting. It is enabled only while something is actually in progress.

The thin **busy bar** below the toolbar animates whenever GitClear is working. Scanning a repository with a very large `node_modules` is the slowest step, because GitClear measures every file in order to report accurate folder sizes.

Buttons grey out to prevent conflicting operations: you cannot start a new search while a deletion is running, and you cannot delete while a scan is in progress.

What GitClear will never delete

GitClear is a deletion tool, so its safety rules are worth stating plainly:

- **Only files Git reports as ignored.** GitClear never decides for itself what is disposable — it asks Git, which honours your `.gitignore` files (including nested ones that override parent rules), `.git/info/exclude`, and your global excludes.
- **Never a tracked file.** Tracked files are not ignored, so they never appear in the tree and cannot be selected.
- **Never your `.git` folder, history, or index.**
- **A whole folder only when Git proves it is entirely ignored.** A folder is deleted as a unit *only* when Git collapses it into a single ignored entry, which guarantees nothing tracked lives inside. A folder that mixes tracked and ignored content is only ever touched file-by-file.
- **Nothing without confirmation**, and everything goes to the Recycle Bin rather than being erased. If an item is too large to recycle, Windows warns you before destroying it and you can back out — see *Very large deletions* below.

GitClear also skips folders it cannot read (permissions, symlinks and junctions) rather than failing, and will not follow symbolic links out of the tree you pointed it at.

Very large deletions

Windows cannot recycle an item that is bigger than the Recycle Bin's configured capacity — it can only delete such an item permanently. A `node_modules` folder of several gigabytes can easily exceed a default Recycle Bin allowance.

GitClear will always ask you first. If something in your selection cannot be recycled, Windows shows its standard permanent-delete warning:

Are you sure you want to permanently delete this folder?

- Choose **No** to back out. GitClear stops immediately and reports *"Deletion stopped — nothing was permanently deleted."* Anything already moved to the Recycle Bin before you declined can still be restored with **Undo**.
- Choose **Yes** only if you are content to lose those items for good — they are **not** recoverable and **Undo cannot bring them back**.

This is the only prompt GitClear lets through mid-operation; ordinary recyclable files are moved without interruption.

If you would rather never see the warning, raise the Recycle Bin's capacity (right-click the Recycle Bin → *Properties*) or delete in smaller batches.

Troubleshooting

"Git was not found on your PATH. Install Git to scan repositories." Install Git for Windows and confirm `git` runs from an ordinary command prompt. Restart GitClear afterwards so it picks up the updated `PATH`.

"No Git repositories found under the selected folder." The folder contains no repositories. Note that GitClear stops descending once it finds a repository, so a repository nested *inside* another repository is not listed separately — select the outer one. Point GitClear at a parent folder if you expected more results.

"No ignored files found in this repository." Nothing is currently ignored, so there is nothing to reclaim. This is common after a clean checkout, or before a first build.

"Could not search that folder: ..." The path does not exist or cannot be read. Check the **Folder** box.

"Git could not scan this repository: ..." Git ran but reported an error — for example a corrupt repository, or a `dubious ownership` complaint on a network or mapped drive. Run `git status` in that repository to see Git's own message.

"Deletion stopped - nothing was permanently deleted." Something in your selection was too large for the Recycle Bin and you declined the permanent-delete warning. Nothing was destroyed; use **Undo** to restore anything that had already been recycled. See *Very large deletions*.

"Some items could not be deleted: ..." Usually a file locked by another program. Close whatever is holding it — a build, an editor, a running app, an antivirus scan — and try again.

The count says thousands of files but deletion was instant. Expected. A wholly-ignored folder is removed as one operation rather than file by file.

A folder is still there after deleting its files. In *mixed* folders GitClear removes the ignored files but leaves the (now possibly empty) folder behind, because it cannot safely assume the folder itself is disposable. It takes no meaningful space.

Limitations and notes

- You cannot drill into a marked wholly-ignored folder, or exclude individual files inside one — it is deleted as a unit or not at all.
- Empty folders may remain in mixed folders after their ignored files are deleted.
- Undo is single-level and is not remembered between sessions.
- GitClear does not remember the last folder you scanned.
- Scan time is dominated by measuring file sizes, so very large ignored trees take a few seconds.